MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick

MASTER'S THESIS

Master graduate: **Roland BÁLINT**

Supervisor: **Conf. dr. ing. Ciprian Pavel OPRIȘA**

September, 2026

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

DEAN,
Prof.Dr.Eng. Vlad MUREȘAN

HEAD OF DEPARTMENT,
Prof.Dr.Eng. Rodica POTOLEA

Master graduate: **Roland BÁLINT**

Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick

1. **Project proposal:** *Brief description of the master's thesis and the initial data*
2. **Project contents:** *(enumeration of the main parts) Example: Presentation page, Title of chapter 1, Title of chapter 2, . . . , Title of chapter n, Bibliography, Appendices.*
3. **Place of documentation:** *Example:* Technical University of Cluj-Napoca, Computer Science Department
4. **Consultants:**
5. **Date of issuing the proposal:** *Example:* November, 2025
6. **Date of delivery:** *Example:* July 5th, 2026

Master's graduate: (*here comes the signature*) . . . . .

Supervisor: . . . . . .

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

Declarație pe proprie răspundere privind autenticitatea lucrării de disertație

Subsemnatul(a) Roland BÁLINT, legitimat(ă) cu ..... seria ..... nr. ....., CNP ....., autorul lucrării Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick, elaborată în vederea susținerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, Programul de studiu ....., din cadrul Universității Tehnice din Cluj-Napoca, sesiunea .....(iulie/septembrie) .....a anului universitar 2024-2025, declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de disertație*.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data

.....

Roland BÁLINT

.....
Signature

Contents

Chapter 1

Introduction

Chapter 2

Objectives of the Research

The objective if to provide a secure, modular and portable architecture for a TOTP authenticator on a USB stick, and a software solution implementing it.

Authenstickator replaces a regular authenticator app like Microsoft's Authenticator or Google Authenticator. For these, you need a mobile phone. If you don't have a dedicated work phone, you might need to tie your personal device to the TOTP verification of a work account. In case you lose your phone, your company might require to wipe your data. Furthermore, mixing personal belongings with work is not a good practice. A USB device is cheap, so it is not an overhead to have a USB device solely for authentication purposes.

While professional solutions like YubiKeys exist, they are quite expesive due to their dedicated hardware and branding. While they are far more secure than a simple USB stick, for two-factor authentication Authenstickator is more than enough.

Chapter 3

Bibliographic Research

This chapter will go through the main ideas, principles, algorithms, and notions used throughout the implementation of Authenstickator. They will be presented as found in literature, with references to bibliographic sources, hence the chapter's title. While the sections don't necessarily come in a given order, I tried to structure them from the main ideas to the more detailed aspects. Each section can be read in isolation, so feel free to jump to the one of your interest.

3.1 Existing Solutions

In this section I will present software solutions (sometimes backed by hardware) which solve a similar problem to what Authenstickator aims to solve. They will be presented in comparison with Authenstickator, so you will see both arguments for and against my solution. Frankly, I found no other project doing exactly what I do, so none of the presented solutions will be a one-to-one match to my project. Also, when comparing, keep in mind that Authenstickator is developed by a single person, in a limited amount of time, and serves more as a proof of concept rather than a polished end product.

Without further ado, lets dive deep into the "opponents" of Authenstickator.

3.1.1 YubiKey

The YubiKey is a product if the company called yubico. It has gained rising popularity over the last few years, both for personal and professional use. It comes in various shapes and forms, but mostly resembles a USB stick. However, these are custom-built hardware designed specifically to support different protocols which need a security key.

Citing from their website [1], "The YubiKey works with hundreds of enterprise, developer and consumer applications, out-of-the-box and with no client software. Combined with leading password managers, social login and enterprise single sign on systems the YubiKey enables secure access to thousands of online services." Their products also support OTP: "OTP supports protocols where a single use code is entered to provide authentication. These protocols tend to be older and more widely supported in legacy applications. The YubiKey communicates via the HID keyboard interface, sending output as a series of keystrokes. This means OTP protocols can work across all OSs and environments that support USB keyboards, as well as with any app that can accept keyboard input."

Their most recommended product is YubiKey 5, and they call it the world's number one multiprotocol security key. From these series, as of August 21st, 2026, the cheapest for 70.18 EUR. Other models from this series go even more expensive, with the biggest price being 102.85 EUR. The models differ in capabilities, but also physical size, the smaller ones costing generally more.

Yubico is a generally trusted company with a good reputation. It is based in Stockholm, Sweden, and was founded in 2007 by Stina Ehrensvard. According to Tracxn[1], it is ranked 1st among its competitors. They have around 300 employees and have a total of 30M USD funding. However, the software that runs in YubiKey is closed-source. They did have some security issues in the past[2], including:

- **ROCA vulnerability - October 2017**. A vulnerability which affected many security keys which implemented RSA. It allowed an attacker to reconstruct the private key by using the public key. Yubico gave free replacements with newer series devices.
- **OTP password protection - January 2018** This affected a specific model, the YubiKey NEO. The password protection for the OTP functionality could be bypassed. Yubico offered a firmware update and free replacements.
- **Cloning vulnerability - September 2024** It allowed a person to clone a YubiKey if they gave physical access. It was remediated through a firmware update. This required advanced cryptographic and technical knowledge: they looked at the electromagnetic emissions of the keys. [2]

In comparison with Authenstickator, YubiKeys are much more secure, since they are specialized hardware. It is backed by a stable company with a team of software and hardware engineers. It went through various iterations, and leads in the domain of security keys. They are a solution for various problems, OTP being just one tiny functionality of theirs. They are widely used, also by big corporations.

In defense of my project, Authenstickator also has some benefits compared to a YubiKey. First, the price of Authenstickator is bond to the price of the USB key it is used on, which is minuscule compared to 70 EUR. Most of us have a spare USB stick laying around without a purpose, and with Authenstickator, we can give life to a potential electronic waste. Second, the source code of Authenstickator is fully open source, so anyone can audit it or reuse it (respecting its GPL-3.0 license). Authenstickator is an academic project, while YubiKeys are professional products.

3.1.2 Virtual FIDO

Citing from the GitHub repository of Virtual FIDO [3]: " Virtual FIDO is a virtual USB device that implements the FIDO2/U2F protocol (like a YubiKey) to support 2FA and WebAuthN. Virtual FIDO creates a USB/IP server over local TCP to attach a virtual USB device. This USB device then emulates the USB/CTAP protocols to provide U2F/FIDO services to the host computer. "

So, this is a fully-software solution for FIDO2 authentication, which is a different protocol than TOTP. However, both FIDO2 (Fast IDentity Online) and TOTP are used

[1] `https://tracxn.com/d/companies/yubico/__fgcOVhxhBthY3y9De52Otl021ENbj9VtVBf2Ho1uoOI#about-the-company`

[2] `https://en.wikipedia.org/wiki/YubiKey#Security_issues`

[3] `https://github.com/bulwarkid/virtual-fido`

for Multi-Factor Authentication (MFA). While TOTP strengthens traditional authentication, FIDO2 eliminates the need for passwords completely.

Looking over the fact that it implements a different protocol, Virtual FIDO is also a solution for MFA. While at first it might sound great that we have this software running on the host machine without an external device, on a second thought you might realize that it completely kills the MFA aspect. It is rather a bypass for FIDO, and should never be used. Possible use-cases could be development and testing, but other than that, the idea is pretty wrong.

So, Authenstickator wins over Virtual FIDO, as Authenstickator tries to provide a security solution, not to bypass one. It serves as a good example that throughout the design of the project, I had to keep in mind that we absolutely want the software (and the secrets) to run/live on the USB, not on the host machine.

3.1.3 USB Raptor

USB Raptor is a free and open-source project designed for Windows, which transforms a USB flash drive into a computer lock and unlock key. Citing from their SourceForge page:[4] " USB Raptor can lock the system once a specific USB drive is removed from the computer and unlock when the drive is plugged in again to any USB port. The utility checks constantly the USB drives for the presence of a specific unlock file with encrypted content. If this specific file is found the computer stays unlocked otherwise the computer locks. To release the system lock user must plug the USB with the file in any USB port. Alternative the user can enable (or disable) two additional ways to unlock the system such is network messaging or password. "

Again, this application is not quite what Authenstickator aims to be, but it also tries to convert a USB stick into a security device. It is not quite a security key, although some websites name it so. The application runs in the background in the system tray using minimal resources. Its behavior is highly customizable, from sound effects to lock delays. It provides backup alternatives if you get locked out, like a RUID file.

After some light research, I came across a Reddit post[5] which askes how to bypass USB Raptor. The poster wants to do so since USB Raptor stopped working on his/her machine. The correct password was not recognized by the software, leading to system lockout. Multiple users report having the same issue, one saying that his/her system had to be wiped. In the same SourceForge page which provides the application, in the Wiki section's comment area I found multiple users asking for unlock keys for exchange of a donation, reminding me of ransomware. The posted source code is also quite outdated, with releases after the last update, so the project cannot be considered fully open source.

It might be paranoia from my side, but based on what findings I described earlier, I would not consider USB Raptor trusted software. While the idea is interesting, it can lead to unpleasant side effects. USB Raptor finds a different way to turn a USB stick into a security device than Authenstickator. I consider my solution to be safer.

[4] `https://sourceforge.net/p/usbraptor/wiki/Home/`

[5] `https://www.reddit.com/r/ComputerSecurity/comments/18ul3ex/how_do_i_bypass_usb_raptor_my_computer_refuses_to/`

3.2 Two-Factor Authentication

"Two-factor authentication is a subset of multi-factor authentication (MFA). It requires two authentication factors to allow access to a service or system. However, two factors from the same category don't constitute two-factor authentication."[3] It is a security concept aimed to reduce the volume of cyberattacks against authentication.

So what are these authentication factors? Authentication is about ensuring the person who says "I am John Doe" is actually John Doe. It all comes down to how he proves his identity. There are several authentication factor types, but the most common are:

1. **Knowledge factor.** Answers the question: what does only the user know? It shall be an information that only one specific person has available. Common examples are passwords, personal identification numbers (PINs), passcodes.

2. **Possession factor.** Answers the question: what does only the user have? Physical possessions, like a driver's license, an identification card, or a mobile device all go into this category. A common scenario is an authenticator application on a smartphone, or push notifications/ SMSes sent to it.

3. **Location factor.** The location of the user at the moment of authentication is used as a restrictive criterion. This is mostly suitable for organizations/ corporations which are active in fixed locations.

4. **Time factor.** Similar to the location factor, but the restrictive criterion being the time of login. Access requests outside a permitted time range are declined and/or reported.

Two-factor authentication is not a new concept. It has been around for quite some time. It has also received criticism, since many doubt their effectiveness. Bruce Schneier, an American cryptographer, computer security professional and adjunct lecturer at the Harvard Kennedy School writes in an article dating back to 2005[4]: "Two-factor authentication isn't our savior. It won't defend against phishing. It's not going to prevent identity theft. It's not going to secure online accounts from fraudulent transactions. It solves the security problems we had 10 years ago, not the security problems we have today." He mentions man-in-the-middle attacks and Trojan malware as two examples which completely bypass two-factor authentication.

Indeed, no security measure is medicine against all cyberattacks. However, there are studies which show a significant increase in security when MFA is enabled compared to when it's not. A study from 2023 conducted by Microsoft employees investigated the effectiveness of MFA in protecting commercial accounts from unauthorized access.[5] They underline that is hard to come with exact numbers in this regard, since compromised users often do not report being compromised. Their article states: "We obtained a sample of 128,000 accounts that had their passwords leaked between April and September of 2022. Users were immediately notified. We retroactively studied those accounts for 30 days prior to the discovery of the credential leak. Reviewing the accounts manually, we found 7,861 accounts that had MFA and for which we could confirm that attackers used the passwords to try to obtain access to protected resources. For those accounts, we found that MFA prevented 98.6% of the attacks."

The same paper mentions a study by Google made in 2019, which found that "challenges and MFA prevented 100% of automated attacks, 96% of bulk phishing attacks, and 76% of targeted attacks." So, while there are ways to circumvent MFA (and implicitly, 2FA), it provides significant security gains especially for big corporations which handle sensitive data. These accounts are often more targeted than personal accounts, and while sophisticated attack methods are out of scope, a big wave of automated attacks and cheap tricks can be solved by two-factor authentication.

3.3 One-time Password Algorithms

"A one-time password (OTP) is a temporary, auto-generated code used to authenticate a user for a single login session or transaction."[6] It is a user-friendly string of characters or numbers, i.e., it can be easily entered manually when prompted to do so. While a traditional, static password is permanent, one-time passwords expire in a short period of time to combat brute-force attacks. They are widely used as a second factor: if it is sent to or generated by a separate device than the one the application runs on, it becomes the second (possession) factor along the regular password (the knowledge factor).

3.3.1 HMAC-Based One-Time Password Algorithm

The HMAC-Based One-Time Password Algorithm, or HOTP for short, is an open algorithm proposed by D. M'Raihi et al.[6], backed by The Internet Society. It was published in 2005. It focuses on defining an algorithm which would have minimal hardware requirements, but be secure. Before its publication, it seems like there were no successful standardization attempts regarding OTP. They mention Java smart cards, USB dongles, and GSM SIM cards, but in today's world, the device is mostly a smartphone.

"The HOTP algorithm is based on an increasing counter value and a static symmetric key known only to the token and the validation service. In order to create the HOTP value, we will use the HMAC-SHA-1 algorithm, as defined in RFC 2104. As the output of the HMAC-SHA-1 calculation is 160 bits, we must truncate this value to something that can be easily entered by a user." They mention the following formula:

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA-1}(K, C)) \tag{3.1}$$

Where *Truncate* represents the function that converts an HMAC-SHA-1 value into an HOTP value, K is the Key and C is the Counter.

Put in simple words, there is a counter which increases with each usage, and a shared secret. By shared, I mean that it is known both by the system which wants to authenticate and the system which validates the authentication, but no one else. The secret is generated by the authentication system and sent to the user once, at a provisioning step. How this is done is out of scope of the specification. The counter and the secret are combined by a hashing function, and converted to a user-friendly format. This HOTP value can be calculated both by the token and the validation service, since their parameters are known by both.

One drawback of this algorithm (considering the problem of sharing the secret solved) is that the counters have to be kept in sync. The paper treats this issue in

[6] `https://www.okta.com/blog/identity-security/what-is-a-one-time-password-otp/`

its Resynchronization of the Counter section. They recommend setting a look-ahead parameter, basically defining a window for the accepted counters server-side. This allows some drifting of the counter. Resynchronization is also an option, but it is "even more difficult than guessing a single HOTP value". Aside of this, like with any other security-related algorithms, parameters have to be chosen with care, such that the algorithm balances between usability and security.

3.3.2 Time-Based One-Time Password Algorithm

Time-Based One-Time Password Algorithm (TOTP) is a derivation of the HMAC-Based One-Time Password Algorithm described in Section 3.3.1. The related RFC is published by the Internet Engineering Task Force in May 2011[7], one of the authors being D. M'Raihi itself (also found in the editors of [6]). As they state, "The HOTP algorithm specifies an event-based OTP algorithm, where the moving factor is an event counter. The present work bases the moving factor on a time value. A time-based variant of the OTP algorithm provides short-lived OTP values, which are desirable for enhanced security."

In other words, instead of the Counter C in Equation 3.1, TOTP uses an integer T which represents the number of time steps between the initial counter time and the current Unix time. The time step X is a system parameter, which represents the time step in seconds, i.e., for how many seconds is a password valid; the recommended default is 30 seconds. The initial counter time $T0$ is the time from which we start counting the steps; the recommended default is 0, so simply the current Unix time is used. For example, if $T0 = 0$ and $X = 30$, $T = 1$ if the current Unix time is 59 seconds, and $T = 2$ if the current Unix time is 60 seconds.

$$TOTP = \text{HOTP}(K, T) \tag{3.2}$$

$$T = \frac{(CurrentUnixTime - T0)}{X} \tag{3.3}$$

Similarly to HOTP, a time window for tolerance can be defined to deal with clock shifts. Naturally, a requirement for TOTP is the ability of the devices to get the current Unix time. With most digital devices, this is not a problem, but clock synchronization is not a trivial problem. "If the time step is 30 seconds as recommended, and the validator is set to only accept two time steps backward, then the maximum elapsed time drift would be around 89 seconds, i.e., 29 seconds in the calculated time step and 60 seconds for two backward time steps." Usually this is more than enough to tolerate clock shifts.

The TOTP algorithm generates the 6-digit codes you are probably familiar with. The users usually download a mobile authenticator application. Figure 3.1 shows three popular authenticator applications, namely, from left to right: Proton Authenticator, Microsoft Authenticator, Google Authenticator. The provisioning step is done by showing a QR code which the user can scan with the application, or the secret may be added manually. After provisioning, the application stores the secret.

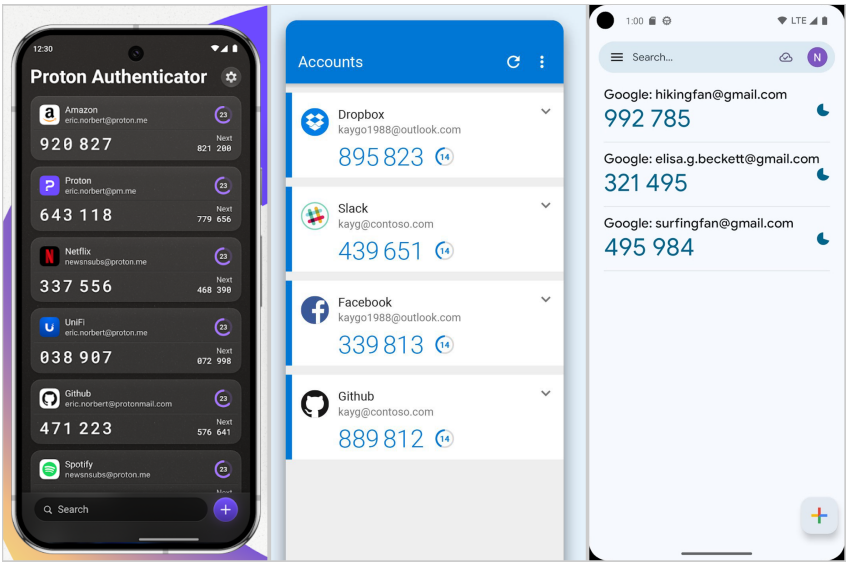

Figure 3.1: Authenticator apps. Source: Google Play.

3.4 Encryption and AES

"Encryption is the process of transforming readable plain text into unreadable ciphertext to mask sensitive information from unauthorized users."[7]. It is a two-way operation: a plaintext can be encrypted into ciphertext, and a ciphertext can be decrypted into plaintext. Encryption and decryption happens with the help of keys (secrets). If the same key is used both for encryption and decryption, the algorithm is a symmetric encryption algorithm. Asymmetric encryption uses two keys: a public key for encryption and a private key for decryption.

Both asymmetric and symmetric encryption algorithms have strenghts and weaknesses. Asymmetric encryption is slower, but more secure since eliminates the need of key exchange. Symmetric is faster, but the key has to be exchanged. Usually (ex: for TLS), asymmetric encryption is used to exchange a symmetric key, and from then on, symmetrix encryption is used (for a given time period, after which a new key is exchanged, the old key becoming expired). Encryption is a must-have for sending sensitive information through untrusted media, like the internet, but also essential to make sure data at rest is protected (file encryption, disk encryption, database encryption).

The Rijndael Cipher is the winner of the AES competition[8]. It is the golden standard of symmetric encryption algorithms. It transforms a plaintext of a given size into a ciphertext of the same size. The plaintext is encoded in hexadecimal, and split into 128-bit blocks. One block can be represented as a 4 by 4 matrix, where each cell contains two hexadecimal characters. Since each hexadecimal character can represent 2 bytes, one cell is 4 bytes, one row is 16 bytes, and the whole matrix is 128 bytes (4 rows).

The AES algorithm repeatedly applies four types of transformations as depicted in Figure 3.2. The figures in this section are from the Rijandael (AES) Animation made by Forma Estudio[9]. This animation explains the algorithm much better than my text, but here is my attempt. The transformations are:

1. **SubBytes**. Uses an S-BOX, i.e., a lookup table. Each cell is substituted (changed) with a value from the S-BOX. The value is chosen by taking the first byte from the

[7] `https://www.ibm.com/think/topics/encryption`

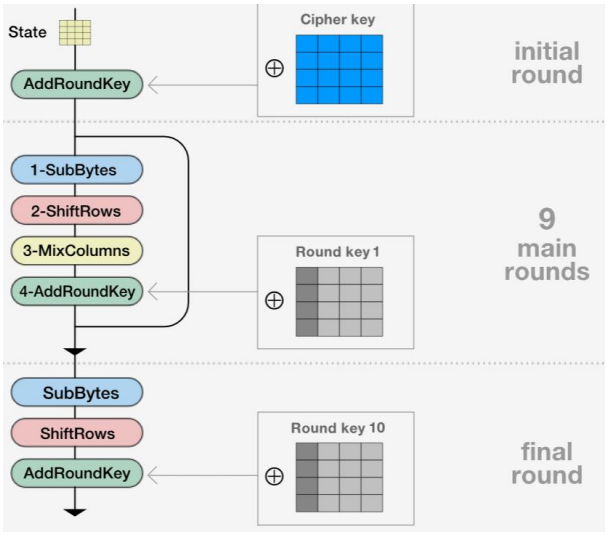

Figure 3.2: AES encryption steps. Source: Forma Estudio.[9].

cell to select the row, and taking the second byte to select the column.

2. **ShiftRows**. Each row is rotated to the left with its index. The first row stays as-is, the second is rotated 1, the third is rotated 2, and the fourth is rotated 3.

3. **MixColumns**. Each column is Galois-multiplied by a specific matrix. Galois-multiplication is a special type of multiplication, which gives results in the same set as its inputs. So this is yet another step to mix bytes up deterministically.

4. **AddRoundKey**. The round key is XOR-ed with the current matrix (explained later).

The **AddRoundKey** step mentions a round key. Round key comes from the Key Scheduling part of the Rijandael algorithm. The cipher key (input of the algorithm) is expanded into eleven partial keys: one for the initial round, nine for the main rounds, and one for the final round. We can visualize the cipher key as a matrix yet again, and the round keys will be generated to its right, column by column (see: Figure **??**). Starting with the cipher key, up until all the columns are generated, the steps below are performed. Note that, the steps marked with ***** are performed only if the column's position is a multiple of 4. For the rest of the columns, only XOR applies.

1. **RotWord***. the column is rotated once, in upside direction.

2. **SubBytes*** Same as for enctyption, using the same S-BOX.

3. **XOR** The column is XOR-ed with the column 4 positions to the left.

4. **Add*** The column is summed with a column from a specific constant matrix (Rcon)

Now the amazing part is that encryption can be done the same way, but by reversing everything. Instead of adding a matrix, we subtract it. Instead of shifting to one direction, we shift to the opposite direction. For substitution, we do the inverse substitution. And Galois-multiplication has its inverse operation as well. While some of these operations look complicated graphically, computers are really good at performing them, making encryption and decryption fast. For its strong security, open-source nature and performance, AES is used in almost all cases when symmetric encryption is needed.

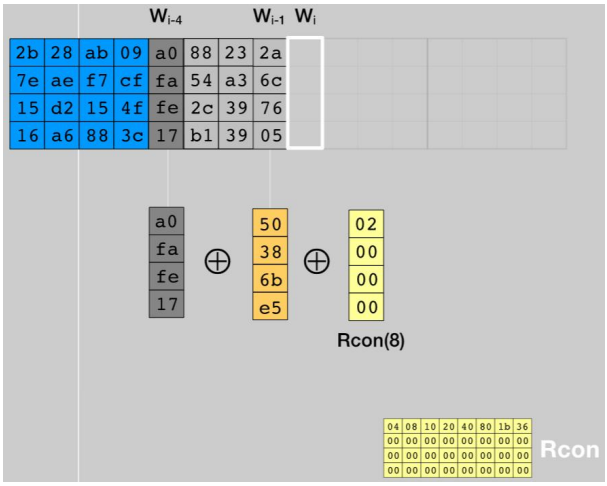

Figure 3.3: AES Key Schedule. Source: Forma Estudio.[9].

3.5 Password-Based Key Derivation. PBKDF2

"In cryptography, a key derivation function (KDF) is a cryptographic algorithm that derives one or more secret keys from a secret value such as a master key, a password, or a passphrase using a pseudorandom function (which typically uses a cryptographic hash function or block cipher)"[8]. They are often used when keys need to be stretched or need to have a certain format. A typical use case is applying password-based key derivation on a key before using it for AES. If the key needs to be of a certain length, depending on the context, just padding it might break the security of the following algorithms, since it is predictable and not random.

Key derivation functions can not only enable random user input to be stretched to the required size, but it usually makes them harder to break as well. These functions are intentionally slow, so that brute-force attacks and dictionary attacks are harder to be performed if inputs are being guessed. If the key derivation is deliberately slow enough, it can be considered a hash function. Such algorithms are, for example, crypt, bcrypt and scrypt.

The Password-Based Key Derivation Functions are examples of key derivation functions. I used the word in plural since there are to major versions of it, PBKDF1 and PBKDF2. "PBKDF1 applies a hash function, which shall be MD2, MD5 or SHA-1, to derive keys. The length of the derived key is bounded by the length of the hash function output, which is 16 octets for MD2 and MD5 and 20 octets for SHA-1. (...) PBKDF2 applies a pseudorandom function to derive keys. The length of the derived key is essentially unbounded."[10]

I will focus on PBKDF2, as even the official publication describing them recommends it over PBKDF1. The PBKDF2 algorithm has the following parameters:

- P. A password, as an octet string (input). This is the password from which we want to derive a key. Usually user input, and we cannot rely on it being of a specific length or "strength".

- S. A salt, as an octet string (input). Just as usually, the salt has the purpose to bring randomness to the output. Even if the same password is provided, given

[8] `https://en.wikipedia.org/wiki/Key_derivation_function`

that the salt is different, the hash will be different. This is a mechanism against brute-force and dictionary attacks.

- c. An iteration count, as a positive integer (input). It serves the purpose of increasing the cost of producing keys from a password. This way it also increases the difficulty of attack, making brute-forcing or building a dictionary take more time. The recommended minimum number of iterations is 1000.
- $dkLen$. The desired length of the derived key (input). This depends on what we want the key to use for later. But as principle, it can be different numbers, so the algorithm is capable of deriving different length keys.
- PRF The underlying pseudorandom function. Depending on which pseudorandom function is used, it might consume different length octets at a time. This length is refered to as $hLen$

First, the number of $hLen$-octet blocks of the derived key is calculated. We simply take the total desired length and divide with the length taken at once, and ceil the result.

$$l = \left\lceil \frac{dkLen}{hLen} \right\rceil \tag{3.4}$$

The number of leftover octets (for the last block) will be denoted with r.

$$r = dkLen - (l - 1) \times hLen \tag{3.5}$$

Each block of the derived key (T_i) will come from the function F.

$$T_i = F(P, S, c, i) \quad \text{for } i = 1, 2, \ldots, l \tag{3.6}$$

And the function F is the XOR of the first c iterations.

$$F(P, S, c, i) = U_1 \oplus U_2 \oplus \cdots \oplus U_c \tag{3.7}$$

Where each iteration U_i comes from applying PRF on the password and different salts. For the first iteration, the salt is S concatenated with an encoded version of i (the block index). For the rest of the blocks, the salt is the result of the previous iteration.

$$\begin{aligned} U_1 &= \mathrm{PRF}(P, S \parallel \mathrm{INT}(i)) \\ U_2 &= \mathrm{PRF}(P, U_1) \\ &\vdots \\ U_c &= \mathrm{PRF}(P, U_{c-1}) \end{aligned} \tag{3.8}$$

After each block is calculated, they are concatenated. From the last block we only take the first r octets. The result is the derived key, having the desired length.

$$DK = T_1 \parallel T_2 \parallel \cdots \parallel T_l\langle 0 \ldots r - 1 \rangle \tag{3.9}$$

3.6 Hashing and Argon2

"Hashing is a process that converts any input to a fixed-length, seemingly random output."[9]. For a function to be called hashing function, it has to fulfill certain criteria, aside of giving the same length output for any length input. First of all, it has to be a one-way function. Unlike for encryption, where we want the ciphertext to be decryptable, in case of hashing, it shall be impossible to convert the output of a hash function back to the input which it came from. Hashes are compared between each other, or used as they are, but never reversed.

Then, it has to have good collision resistance. Since we convert to a fixed size string, multiple inputs will give the same output, inevitably. It is like parking infinite cars, in seemingly random order, into finite parking lots. At one point, parking lots will fill up, and two cars will have the same lot. This means that, if used for password storage and authentication, there is a mathematical chance that two passwords give the same hash, even with salting. This would mean logging in with an incorrect password. The catch is that for hash functions with good collision resistance, the chances are terribly low. For example, for the MD5 algorithm, which was broken in 2004 (it was shown that it is not collision-resistant), the probability to have two passwords producing the same hash is $\frac{1.47}{10^{29}} = 0.00000000000000000000000000147\%$.[10].

The third criterion for a hash function to be considered usable is to be deterministic, not random. That is why, in the last paragraph's analogy, I said "seemingly random". The same input must give the same output all the time. Otherwise, it would just be a random number generator, without the current practical use cases.

Hash functions are used for various purposes, out of which the most common are:

- **Password storage.** Databases are often leaked, so storing the hash of a password instead clear text is the 101 of cybersecurity. In this case, it is extremely important to use a strong hashing algorithm, with proven collision resistance.

- **Fast look-up data structures.** Hash maps, for example, are often used where data has to be retrieved in `O(1)` time complexity. If we want to look for something, we compute its hash, which is the address it is supposed to be at. This way, we can instantly look up data. Collision resistance in this use-case is important for maintaining the speed advantage, not for security.

- **File signatures.** A hash function can take the contents of a whole file and convert it into a much shorter hash. This way, we get a unique value, a signature for each file (again, given that no collisions happen). We can easily check if a file changed, by looking at the previous and current signature. File signatures are also used for static malware analysis: we can take the hash of a program, feed it to a database like `https://www.virustotal.com`, and check if it is a reported malware.

Argon2[11] is a password hashing algorithm, the winner of the Password Hashing Competition. "PHC ran from 2013 to 2015 as an open competition—the same kind of process as NIST's AES and SHA-3 competitions, and the most effective way to develop a crypto standard. We received 24 candidates, including many excellent designs, and

[9] `https://www.bitdefender.com/en-us/business/infozone/what-is-hashing`
[10] `https://infosecscout.com/two-passwords-same-hash/`

selected one winner, Argon2, an algorithm designed by Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich from University of Luxembourg."[11].

The algorithm aims to maximize the cost of password cracking on Application-Specific Integrated Circuits (ASICs). An ASIC is An "integrated circuit designed for a specific customer, application, or market (...) interconnected, and simulated to provide the desired system functions and level of performance." [12]. In this context, it refers to hardware specialized for password cracking. The advanced attackers who use such software often exploit parallelization (on GPUs, FPGAs, with more cores), they compute blocks ahead, they juggle with time-memory tradeoff ("the adversary can trade memory for time and do his job on fast hardware with low memory."[11]).

Argon2 takes all these aspects into account, and comes with a solution in two flavors: Argon2d and Argon2i. Argon2d is faster and uses data-depending memory access, suitable for cryptocurrencies, for example. Argon2i is slower and uses data-independent memory access, which is preferred for password hashing. Data-dependent indexing takes into account the password when choosing the memory index to write to, this way making it impossible to prefetch and precompute missing data. Some software, like Bitwarden, use Argon2id, which uses "a combination of data-depending and data-independent memory accesses, which gives it some of Argon2i's resistance to side-channel cache timing attacks and much of Argon2d's resistance to GPU cracking attacks."[13]

The mathematics behind Argon2 are out of scope for this document, but simply said, the algorithm consists of three steps. First, memory initialization fills a defined memory size with pseudorandom values derived from the password, salt, and other parameters. Then, the memory's contents are repeatedly combined with the password and the salt. After all the iterations, the used memory is compressed into a fixed-size output (the hash).[14]. All this makes Argon2 an outstanding choice for password hashing, proven to be up-to-date with today's attackers.

3.7 Full Disk Encryption and LUKS

Full disk encryption, as its name suggests, refers to encrypting every byte of a physical disk. This mechanism is especially powerful since it provides a layer of security for the whole contents of a machine. It also works for removable media. With full disk encryption, an attacker's physical access does no longer mean compromised data. With powerful encryption, brute-forcing decryption of a full disk takes so much time that it is virtually impossible. "LUKS (Linux Unified Key Setup) is the standard disk encryption format for Linux, built on top of the dm-crypt kernel module. It is the encryption layer used by default when you select "encrypt disk" during installation on Fedora, Ubuntu, Debian, and most other major distributions."[15].

It uses PBKDF2 to enhance weak user passwords and has a two level key hierarchy.[12] This means that a master password is generated, which is not shared with the user, and the user's password is used to encrypt this master password, which is then stored in a slot. There are several such slots (key material sections) in the LUKS header, meaning

[11] https://www.password-hashing.net/

[12] https://www.analog.com/media/en/analog-dialogue/volume-24/number-3/articles/volume24-number3.pdf#page=12

[13] https://bitwarden.com/help/kdf-algorithms/

[14] https://jumpcloud.com/it-index/what-is-argon2

[15] https://secure-os.org/articles/luks-encryption/

that multiple passwords can be used to decrypt the disk. As a result, one can have a recovery key along the usual user password.

The LUKS header is the first few bytes of the disk which contains metadata needed for decryption (the algorithm, the KDF etc.). It is a good idea to have a periodic backup of the LUKS header, since if this part of the memory is damaged, the whole disk becomes inaccessible. It is a single point of failure, but there is no free meal.

LUKS2, the newer variant of LUKS, is a good option for encrypting disks and USB devices. As with other cryptographic algorithms, there have been vulnerabilities in their implementation: in 2016, you could gain access by simply holding the enter button.[16]. But it is a good standard, and its open-source implementation is patched regularly.

3.8 TPM

"A Trusted Platform Module, also known as a TPM, is a cryptographic coprocessor that is present on most commercial PCs and servers" [13].

3.9 Attack Types

3.10 Testing Principles

3.11 Programming Patterns

[16] `https://hmarco.org/bugs/CVE-2016-4484/CVE-2016-4484_cryptsetup_initrd_shell.html`

Chapter 4

Project Presentation

4.1 The Proposed Framework

The goal of this Masters' thesis is to provide a generic framework for a TOTP application on a USB stick, as well as an implementation for it. In this section, we will focus on the framework, i.e., the ideas and principles, rather than the technology used. The presented framework could work on other devices as well, it is a generic flow of actions meant to be abstract. At the same time, the framework's main goal is security, so it has to take into account all kinds of attacks and countermeasures for them. With that in mind, the framework focuses on minimizing the possibility of exploits, errors, deadlocks.

Simply put, a TOTP application generates TOTP codes (numbers) based on TOTP secrets (base64 codes). The application which requires TOTP (example: Gmail, Facebook, etc.) displays a QR code or an equivalent secret. The user scans the QR code / introduces the secret into the TOTP application. From then on, the TOTP application stores that secret in a way or other, and keeps generating TOTP codes.

So the problem can be reduced to the following aspects/ goals:

1. **Generate the TOTP codes**: This is the main functional requirement of the framework. TOTP code generation is a standard algorithm and requires only the current time and the TOTP secret.

2. **Store the secrets safely**: Store TOTP secrets in a way which is secure and handle possible attack scenarios.

3. **Keep the 2FA aspect**: The framework treats the device on which the app runs as the possession factor of Two-Factor Authentication 3.2. Breaking this aspect would make the framework useless.

Figure 4.1 shows the two main flows of the proposed architecture. The following sections will go in depth on each aspect.

4.1.1 USB Encryption

The main security layer is provided by the device on which the app resides: the USB itself shall be encrypted. For Linux, a password-protected LUKS2-encrypted USB stick shall be used. The encryption of the device applies on its whole content. This ensures that the stored secrets cannot be read without providing the password. The passwords could be saved in plaintext, the USB encryption would encrypt it anyway.

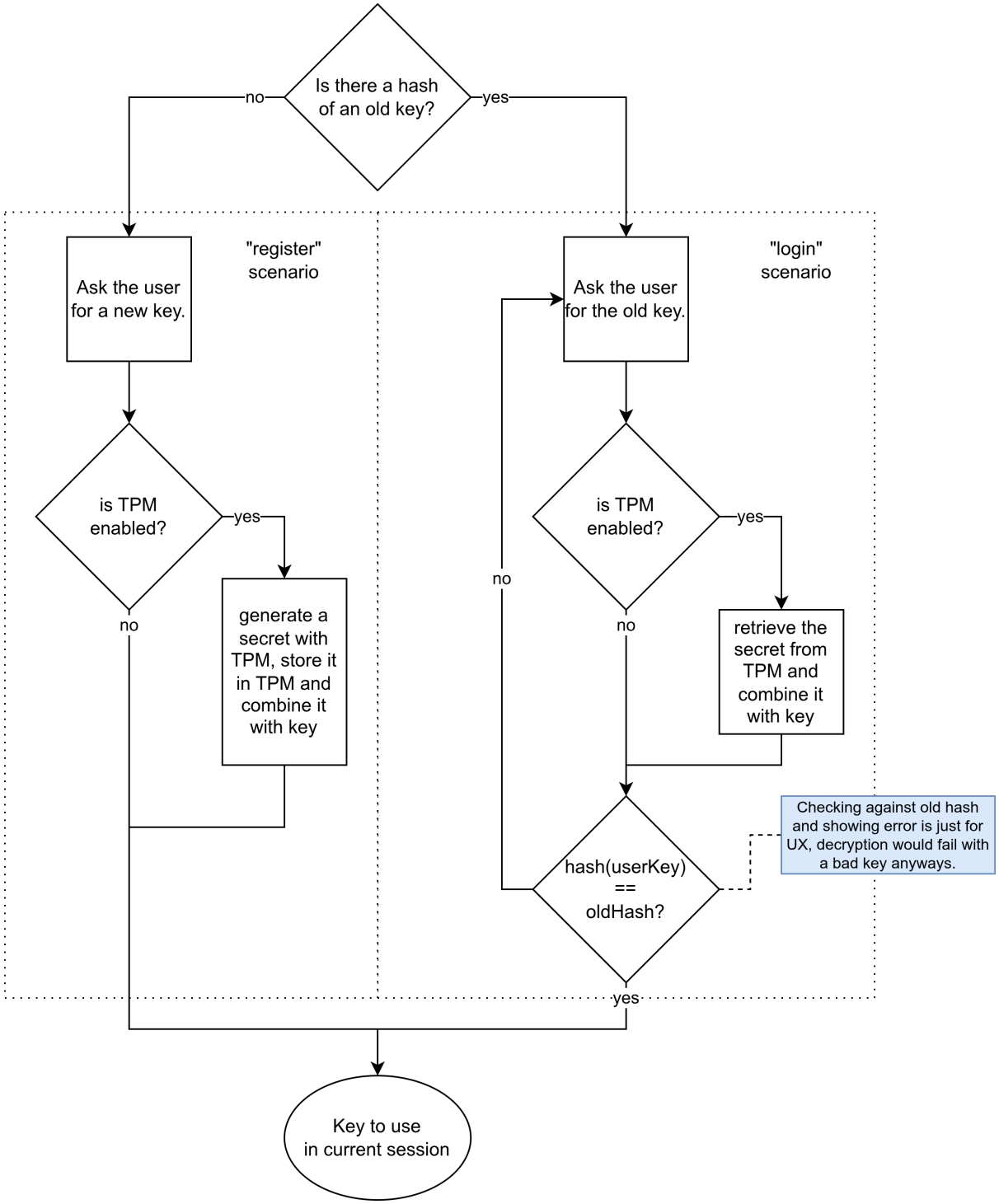

Figure 4.1: The two main flows of the proposed framework.

The main issue of USB sticks (in our context) is that they are not read-only media. Someone could steal the USB stick without our knowledge, and copy its contents. Stealing and putting back the USB stick could be done in less than a minute, making this kind of attack easy to execute. The attacker could then brute-force breaking the USB without any time limitations. Or even worse, if the attacker gets to know our password, he/she can break in without having our media, using the copy. Nevertheless, USB Encryption is a good first line of attack against not-so tech-savvy attackers.

4.1.2 User Password

Aside of the password for the USB's encryption, the framework also requires a user password. It is recommended for this password to be different from the password provided for the USB's encryption, but a check for difference is out of scope. So it is up to the user what password it chooses for the application. We will reference to this as the User Password from now on.

You might ask, what is the purpose of the User Password? It is used to encrypt the internal storage of the application, i.e., the secrets. While the application's source code, including the storage, is encrypted at rest due to USB Encryption, this encryption ensures the passwords are encrypted even when the app is running. This protects against "coffee-shop" attacks: the user might leave their device unlocked and unsupervised, and an attacker could quickly read the secrets from their device. There are other use-cases of the user password as well, described in later sections.

The user password is not stored. Instead, the hash of the user password is stored inside a file. If this file does not exist, it is considered that no user password was provided beforehand, so the user is asked to enter a new password. If this file does exist at start, the user shall provide the old password, which is verified against the existing hash. An incorrect password prevents the start of the flow, since the system has no knowledge about the password, and without it, it cannot decrypt the storage, to access the stored secrets.

4.1.3 TPM

The read-write nature of a USB cannot be changed, and making a USB stick uncopiable is either impossible or out of scope. The encryption with the help of the user password suffers the same vulnerability as the USB encryption, it is just another layer, but breakable in the same fashion. Detecting if another TOTP Application is using the secret (for example, if the attacker stole the secret and introduced it into a TOTP application) is also out of scope. But, something had to be done against the attack scenario presented in Section 4.1.1. This is where the TPM chip comes into discussion, and where we start to exchange convenience for security.

Our framework uses the TPM to get a secret which only one machine can provide, and mix it with the user-provided key. How the secret is mixed with the user password is up to the encryption algorithm. The most straight-forward option, and also the one implemented by Authenstickator, is to use the secret as a salt.

Initially, this secret does not exist. It has to be random, unpredictable, and long. So, the TPM is used to generate it. This provides hardware-backed true randomness. The secret is then stored inside the TPM's NVRAM. This is non-volatile memory, meaning

that it will be present after reboot. Upon each start, the NVRAM is queried for the secret.

Due to the nature of the TPM chip, the random number cannot be guessed. It is 16 bytes long, so brute-forcing it is not feasible. Combining with the user password makes it impossible for other applications using the TPM (for example, malware) to decrypt the storage. If the combination is properly made (ex: secret used as salt for the key), brute-forcing the decryption will also not be feasible.

The downside of using TPM is the same as the main upside: it ties the application to a specific machine. If the user uses one machine all the time, it is not a problem. If the user switches between multiple devices (which support USBs), he/she might consider not using TPM. The framework marks TPM usage optional: if disabled, simply the user password is used for storage encryption.

4.2 The Implementation

The implementation of the proposed framework presented in Section 4.1 is the application called Authenstickator. It consists of a Python front-end and a simple web front-end (HTML, CSS, JavaScript). The architecture of Authenstickator can be seen on Figure 4.2. The front-end and the back-end is decoupled, resulting in a modular architecture. In the following, detailed description of both will come.

4.2.1 The Back-end

As mentioned earlier, the back-end of the application is written in Python. While it might sound like a weird choice for a security-first application, there are reasons for this decision. First and foremost, Python provides a lot of libraries, making implementation easier. Using standard implementations and not reinventing the wheel is usually the better approach for small teams; and especially in the case of cryptographic algorithms, like encryption and hashing.

Second of all, Python provides an easy way to manage dependencies. With the help of the `pip` package manager, any library can be easily downloaded. All the libraries were downloaded inside a virtual environment (`venv`), so they are isolated and do not mess with the system's packages. The downloaded packages were then listed inside a `requirements.txt` file, so one can install all of them with a single command.

While Python isn't famous about its OOP capabilities, I chose an object-oriented approach. For me and for most developers, it is simply a thing of habit to think of complex problems in terms of classes, objects, and interactions between objects. It provides structure, and a common language among developers. In Authenstickator, every Python file contains exactly one class, kind of like how Java enforces it.

Python is also duck-typed, meaning that for Python, "if something quacks, it is a duck". It does not enforce types, there is no compilation. However, types can be linted, and I naturally chose to do so. More specifically, in each method's signature, I also specified what data type I expect for each parameter, and what the method should return. This is not enforced by Python, but a good IDE (like PyCharm) can catch a lot of bugs this way. Also, it is clearer to understand the intentions of methods.

The backend consists of several packages, which will be presented in the next sections. Each package is independent of the other. They can be (and are) unit tested; see: Section 5. They will be presented in the following.

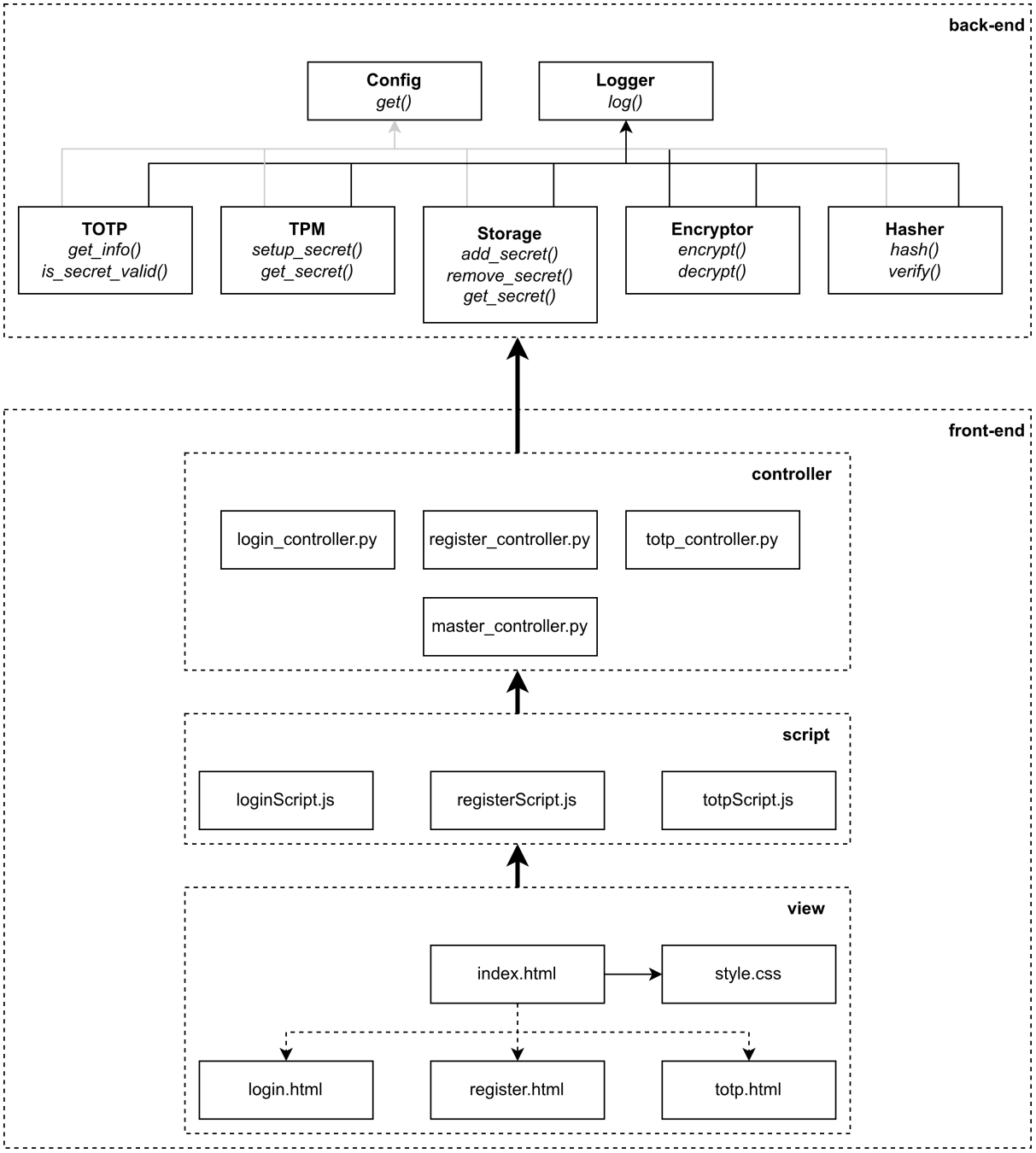

Figure 4.2: High-level diagram of the architecture.

Config

The Config package contains the `ConfigManager`, as well as one or two JSON files. The `ConfigManager` is a singleton class, and its instance provides the `get` method. The role of this class and its method is to read the JSON configuration file and provide configuration values for other back-end classes. This way, the developer (or a more tech-savvy user) can easily change the parameters (configs) of Authenstickator.

The name of the config file is hardcoded, and it is `config.json`. It provides a multi-layer config file, where configs are grouped by the package/ feature they are used by. The developers also have the possibility to define a `config-local.json` file, which if present, has priority over `config.json`. This is also part of `.gitignore`, i.e., not in version control. So it is solely for local development, avoiding pushing to git test configurations.

An example `config.json` looks like this:

```
{
  "logger": {
    "log_level": "DEBUG",
    "log_file_path": "log.txt"
  },
  "pywebview": {
    "debug": false
  },
  "tpm": {
    "enabled": true,
    "fapi_profile_name": "P_RSA2048SHA256",
    "fapi_profile_file": "P_RSA2048SHA256.json",
    "fapi_profile_url": "https://raw.githubusercontent.com/
     tpm2-software/tpm2-tss/3.2.0/dist/fapi-profiles/
     P_RSA2048SHA256.json",
    "virtualized": false,
    "virtualized_tpm_path": "/tmp/vtpm",
    "virtualized_tpm_port": 2321,
    "virtualized_tpm_ctrl_port": 2322,
    "virtualized_tpm_startup_seconds": 1
  },
  "storage": {
    "storage_file_path": "storage.txt",
    "hashed_user_password_file_path": "hashed-user-password.
     txt"
  },
  "encryptor": {
    "type": "AES"
  },
  "hasher": {
    "type": "ARGON2"
  }
}
```

The meaning of each configuration parameter will be explained in the corresponding package's section.

Config Name	**Allowed Values**	**Meaning**
`logger.log_level`	`"ERROR"`, `"WARNING"`, `"INFO"`, `"DEBUG"`	The lowest log level which shall be logged.
`logger.log_file_path`	A valid path, like `"log.txt"`	The path at which the log file shall be saved.
`pywevview.debug`	`true`, `false`	If pywebview should run in debug mode or not.
`tpm.enabled`	`true`, `false`	If the TPM (and related functionalities) shall be used.
`tpm.fapi_profile_name`	todo	todo
`tpm.fapi_profile_file`	todo	todo
`tpm.fapi_profile_url`	todo	todo
`tpm.virtualized`	todo	todo
`tpm.virtualized_tpm_path`	todo	todo
`tpm.virtualized_tpm_port`	todo	todo
`tpm.virtualized_tpm_ctrl_port`	todo	todo
`tpm.virtualized_tpm_startup_seconds`	todo	todo
`storage.storage_file_path`	todo	todo
`storage.hashed_user_password_file_path`	todo	todo
`encryptor.type`	todo	todo
`hasher.type`	todo	todo

Logger

The logger package contains `LoggerManager`, yet another singleton class. As its name suggests, the logger manager provides a logger instance for logging. The standard `logging` library is used. For simplicity, Authenstickator uses only four log levels. The log level can be set by changing the value of the config `logger.log_level`.

- **Error**: for fatal issues, ones which break execution.
- **Warning**: for issues which should not be ignored, but do not crash the app.
- **Info**: for messages which are not issues, but things that might help in audit.
- **Debug**: the most low-level messages; these are useful only for development.

While the logger is centralized, logging happens in two places: the console and the log file. The same log lines are printed in both outputs, but the log file persists, meaning that new logs are appended to the old ones instead of starting a new file at each run. This is good for debugging and audit.

The format of log lines is: date and time, severity level in a letter (E, W, I, D), the file and the line of the source, the logger, and the message itself. For example:

```
[2026-08-16 22:14:47] [E] [util.py:208] [pywebview] Error while
   processing window.native.bin: unable to get the value
```

While there is only one logger instance, `pywebview`, the front-end library also logs its warnings and errors. These are useful, so instead of throwing them out, they are "propagated" to our logger. This is why the logger is also printed in the log line: it can be either `[root]` or `[pywebview]`.

As a convention, each class instantiates a logger instance outside its constructor, for simplicity. Methods which handle requests from the UI often log on debug level entrance in them. As a general rule, secrets and sensitive data shall not be logged. Logging such information would make encryption of the storage broken.

TOTP

TODO

TPM

TODO

As explained in [14], on Linux, TPM shows up as device nodes. `/dev/tmp0` is the Hardware Interface, while `/dev/tpmtm0` is the Kernel Resource Manager. Most applications use the Kernel Resource Manager, since through that entry the kernel handles context management and other low-level mechanisms.

TSS stands for Trusted Software Stack, and it is the software around the TPM chip. It has a layered architecture, providing programmer-friendly API calls. Its layers are specifications, and there are different implementations of it. The four layers are:

1. **FAPI**: Feature API. High-level, developer-friendly. No need to handle sessions and authorization.
2. **ESAPI**: Enhanced System API. Wraps sessions and handles retries when TPM is busy.
3. **SAPI**: System API. Low-level C interface implementing the tmp2.0 specifications.
4. **TCTI**: TPM Command Transmission Interface. The Transport Layer, working with bytes.

Authenstickator uses the `tpm2_pytss` library and the FAPI layer. FAPI was chosen since it is the most high-level, so the resulting code will be more generic. Going lower in the TSS layers would mean more OS/ hardware specific code, and more chances to make errors.

When this layer is used, configurations are held in a file named `fapi-config.json`. The default location is `/etc/tpm2-tss/fapi-config.json`, but it can be overwritten by setting the `TSS2_FAPICONF` environment variable. Authenstickator generates one such file specifically for the usage of this app, so that it does not mess with system-wide TPM settings. It might look something like this:

```
{
    "profile_name": "P_RSA2048SHA256",
    "profile_dir": "/home/broland/.local/share/
     authenstickator/profile-dir",
    "user_dir": "/home/broland/.local/share/authenstickator/
     user-dir",
    "system_dir": "/home/broland/.local/share/authenstickator
     /system-dir",
    "system_pcrs": [],
    "log_dir": "/home/broland/.local/share/authenstickator",
    "ek_cert_less": "yes",
    "tcti": "swtpm:port=2321"
}
```

The FAPI profile is downloaded from the internet, from the URL specified in the config file. The profile's version shall match the version of the installed native `tpm2-tss/libtss2-fapi`. This can be retrieved with the help of the command `apt policy libtss2-fapi1`. Note that this is different from the `tpm2-pytss` version shown in `requirements.txt`.

Storage

TODO

Encryptor

The AES algorithm is a symmetric one, meaning it uses the same key for encryption and decryption. It does not accept any key, it has to be 16, 24 or 32 bytes long. Requiring the user to choose a key with a fixed length is tedious, so Password-Based Key Derivation Function 2 (PBKDF2) is used. This mixes its input with a salt, and outputs a fixed length string, making it perfect for our use case.

TODO

Hasher

TODO

4.3 Provisioning the TPM: a Requirement for the Application

While the application is bundled with its dependencies, the usage of TPM comes with a cost. These chips need to be provisioned, which is a one-time operation to make them run correctly. Provisioning a TPM should be an easy process, but it is rather difficult in practice. Since provisioning might require authentication, it cannot be done by an automatic script. Also, if you mess too much with your real TPM, you might get locked out, just like I did.

If you run a Debian-based distribution, you might try installing with `apt` the `tpm2-tools` package. After installation, try running the `sudo -u tss tss2_provision`

command. It shall create a default FAPI configuration file at path `/etc/tpm2-tss/fapi-config.json`, and automatically download some profiles in `/etc/tpm2-tss/fapi-profiles/`. For more details about this JSON, see Section 4.2.1.

Unfortunately, the `tpm2-tools` package from `apt` did not work for me, and might not work for you neither. If this is the case, you can try running the `setup_fapi.sh` script.

4.4 Development

Development was done on a Linux machine, namely on Pop!_OS 22.04 LTS. The development environment described below was tested on a virtual machine with a fresh Ubuntu 22.04.5 LTS installation. As a result, the following commands will probably work on Debian-based Linux distributions. The IDE used throughout development was PyCharm IDE, but other editors shall work as well.

4.4.1 Basic Setup

First, ensure that your system has Python. Use the following command:

```
python3 --version
```

Both the development machine and the test VM used Python 3.10.12, which came with the OS installation. So, if you have Python-related issues, try this specific version.

Then, Git is used for version control. The source code of Authenstickator is available on GitHub as a public repository, and we will clone the project from there.

```
sudo apt install -y git
git clone https://github.com/broland29/authenstickator.git
cd authenstickator
```

Next, you will need to create a virtual environment and activate it. A virtual environment (`venv`) is used to isolate Python packages used for Authenstickator from other Python packages on the system. This is standard dependency management, and ensures not just that Authenstickator works well, but that we don't break other apps. Each developer creates its own virtual environment and these are not pushed to git (specified in `.gitignore`).

```
sudo apt update
sudo apt install -y python3.10-venv
python3 -m venv venv
source venv/bin/activate
```

Before installing the Python dependencies from `requirements.txt`, there are other Linux packages required for the Python packages to work. These include: a build compiler, the TPM2 development headers and build tools, some FAPI requirements (TPM-related), and some `PyGObject` requirements (`pywebview`/ UI related). For development, to avoid TPM lockout, a software TPM, `swtpm` is recommended, which also needs installation. Be careful with the following command: if you accidentally leave spaces after the line-breaking backslashes, it won't work.

```
sudo apt install -y git \
    libtss2-dev pkg-config python3-dev \
```

```
    libssl-dev libjson-c-dev libcurl4-openssl-dev \
    libgirepository1.0-dev libcairo2-dev libglib2.0-dev gobject-
       introspection \
    swtpm
```

Now that we have the dependencies ready, we can install the actual Python libraries Authenstickator needs. Since we have the virtual environment activated (signaled by the `(venv)` prompt), the libraries will be installed only inside this environment. The requirements are listed in `requirements.txt` and can be installed with the following command:

```
pip install -r requirements.txt
```

Once it is done installing, the app can be started. Remember that, for subsequent launches, you will also need to activate the virtual environment before starting the app.

```
python3 -m src.main
```

4.4.2 Testing with TPM on a VM

If you wish to test the application with the real TPM setting, but still want to avoid using your own TPM and getting locked out, you can use a Virtual Machine. My recommendation is QEMU/KVM, since it is easy to set up and works well. The following steps for TPM setup are for QEMU/KVM 4.0.0. Turn of the virtual machine if it is currently running. Inside the "Show virtual hardware details" panel (the icon containing the letter I, when the VM is selected), click "Add Hardware". Select TPM. Inside "Advanced options", select Model "CRB" and Version "2.0". Click finish, and start the VM.

4.4.3 Setup TPM

First, check if you actually have a TPM chip with the following commands:

```
sudo dmesg | grep -i tpm
ls -l /dev/tpm0 /dev/tpmrm0
```

If the output contains log files like below, good news, you have a TPM chip and can move to the next step.

```
(venv) broland@broland:~/authenstickator$ sudo dmesg | grep -i tpm
[sudo] password for broland:
[    0.012065] ACPI: TPM2 0x000000007FFD2C3C 00004C (v04 BOCHS  BXPC     00000001 BXPC 00000001)
[    0.012076] ACPI: Reserving TPM2 table memory at [mem 0x7ffd2c3c-0x7ffd2c87]
[    2.043231] tpm_crb MSFT0101:00: Disabling hwrng
(venv) broland@broland:~/authenstickator$ ls -l /dev/tpm0 /dev/tpmrm0
crw-rw---- 1 tss root  10,   224 aug 20 18:30 /dev/tpm0
crw-rw---- 1 tss tss  253, 65536 aug 20 18:30 /dev/tpmrm0
```

Figure 4.3: The `dmesg` logs contain such lines if TPM is present.

Unfortunately, the `tpm2-tools` package from `apt` does not work. You can try your luck with these commands though.

```
sudo apt install -y tpm2-tools
sudo -u tss tss2_provision
```

For me, this did not work, I got a command not found error. The alternative is to compile `tpm2-tools` from source. It is very fast and shall work out of the box.

```
sudo apt install -y build-essential autoconf autoconf-archive
   automake libtool pkg-config gcc git libssl-dev libcurl4-
   openssl-dev libjson-c-dev uuid-dev libtss2-dev
cd ~
git clone https://github.com/tpm2-software/tpm2-tools.git
cd tpm2-tools
./bootstrap
./configure --enable-fapi
make -j$(nproc)
sudo make install
sudo ldconfig
sudo -u tss tss2_provision
```

The current user has to be added to the tss group.

```
sudo usermod -aG tss "$USER"
```

Development was done with a simulated TPM. Initially I worked with the real TPM, and I got locked out when I was messing with it. A virtualized TPM makes experimenting easier and leaves the real TPM untouched.

The TPM emulator `swtpm` can be downloaded with APT:

```
sudo apt update && sudo apt install swtpm
```

It can be started by creating a temporary directory (will be deleted at system shutdown) and using it to run the virtual TPM. After this, the third command sets the variable `TSS2_TCTI`, so that in the current terminal session the virtualized TPM replaces the real one.

```
mkdir -p /tmp/vtpm

swtpm socket --tpm2 --tpmstate dir=/tmp/vtpm --ctrl type=tcp,port
   =2322 --server type=tcp,port=2321 --flags not-need-init &

export TSS2_TCTI="swtpm:port=2321"
```

To check manually if swtpm is running, you can netcat it.

```
nc -zv localhost 2322
```

If anything goes wrong, the virtual TPM can be stopped easily by killing its process, and the folder can be deleted.

```
pkill swtpm | rm -r /tmp/vtpm
```

FAPI needs a one-time initialization, called provisioning. This can be done with the `provision` method. To avoid error `0x0006001f` (`TSS2_FAPI_RC_PATH_ALREADY_EXISTS`), this method is called with parameter `is_provisioned_ok` set to true. This way the application does not fail on subsequent startups, or if the user provisioned the TPM beforehand manually or through another application.

The standard development starts with cloning the project.

To use `tpm2_pytss` and its `FAPI` component, you need to install a build compiler; the TPM2 development headers and build tools; some FAPI requirements; and some PyGObject requirements, which are dependencies for pywebview. For the software TPM, swtpm is needed.

```
sudo apt install git \
    python3.10-venv \
    build-essential \
    libtss2-dev pkg-config python3-dev \
    libssl-dev libjson-c-dev libcurl4-openssl-dev \
    libgirepository1.0-dev libcairo2-dev libglib2.0-dev gobject-
       introspection \
    swtpm
```

Careful with trailing whitespaces afet that might break your command.

```
git clone https://github.com/broland29/authenstickator.git

cd authenstickator

python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt

python3 -m src.main
```

To ensure that the app is properly working, you can run the tests:

```
python -m pytest
```

The app is run as a module (hence the `-m` flag) and started from the root folder. This means that imports have to start from the `src` folder, for example, `from src.config.config_mana import ConfigManager` instead of `from config.config_manager import ConfigManager`. While the latter might work in PyCharm due to it modifying the `PYTHONPATH` in the back.

If you end up adding new dependencies, run:

```
pip freeze > requirements.txt
```

Common errors: `tpm2_pytss.TSS2_Exception.TSS2_Exception:  tcti:IO failure`: you are probably running with virtualized TPM enabled, but the virtualized TPM is not running.

The command to create a USB export:

We have to specify paths to point to ., so that imports work correctly. We put the dist/build/spec paths to the temporary folder, to not clutter the project folder (messes with git and such). Spec path is not set, so the .spec file is generated in the default location, where it is executed (in the project root). This is intentional, since add-data will take paths relative from the .spec file. JSON and HTML files are not added by default and need to be added with add-data. See pyinstaller –help for more details.

```
pyinstaller src/main.py \
    --clean \
```

```
    --add-data "src/config/config.json:src/config" \
    --add-data "src/ui/view:ui/view" \
    --add-data "src/ui/script:ui/script" \
    --paths . \
    --distpath /tmp/authenstickator/dist \
    --workpath /tmp/authenstickator/build
```

There is always a key, no matter if encryption or tpm is disabled.
Max line length is 100 characters.
Making real TPM work: 1. Make sure you have a TPM2 chip:
sudo dmesg | grep -i tpm
2. Add your user to the tss group.
sudo usermod -aG tss ”$USER”
3. Restart.
4. Run the app (provisions in the back)
Testiing with VM: - start from fresh install - add to group - add TPM - add stick

Chapter 5

Theoretical and Experimental Results

5.1 Testing on Linux

Since the most popular distro is Ubuntu[1], I chose to test and build on a Ubuntu 22.04 LTS machine. Ubuntu 20.04 LTS hit its Standard Security Maintenance deadline at May 2025, while 22.04 LTS is still in free support until May 2027. Distros shall be backward compatible, so the results shall be the same for newer Ubuntu releases as well, and the setup will probably work on Debian and other Debian-based distros as well.

5.2 Unit tests

For unit tests, the `pytest` library was used. To run all tests, with the virtual environment activated, run the following command:

```
python -m pytest
```

[1] 2025 Stack Overflow Developer Survey: `https://survey.stackoverflow.co/2025/technology`

Chapter 6

Conclusions

Bibliography

[1] yubico, "How the yubikey works," https://www.yubico.com/products/how-the-yubikey-works/, accessed: August 21, 2026.

[2] T. Roche, "Eucleak side-channel attack on the yubikey 5 series (revealing and breaking infineon ecdsa implementation on the way)," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 4026–4043, also available online: https://ninjalab.io/wp-content/uploads/2024/09/20240903_eucleak.pdf.

[3] Fortinet, "What is two-factor authentication (2fa), and how can it be enabled?" https://www.fortinet.com/resources/cyberglossary/two-factor-authentication, accessed: August 8, 2026.

[4] B. Schneier, "Two-factor authentication: too little, too late," *Communications of the ACM*, vol. 48, no. 4, p. 136, 2005.

[5] L. Meyer, S. Romero, G. Bertoli, T. Burt, A. Weinert, and J. Lavista Ferres, "How effective is multifactor authentication at deterring cyberattacks?" 05 2023.

[6] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-Based One-Time Password Algorithm," RFC 4226, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/info/rfc4226/

[7] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, May 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6238/

[8] National Institute of Standards and Technology (NIST), "Advanced encryption standard (aes)," U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 197-upd1, May 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf

[9] F. Estudio, "Rijndael (AES) animation," 2007, originally made in 2004, enhanced for the CrypTool project. [Online]. Available: https://formaestudio.com/rijndaelinspector/archivos/Rijndael_Animation_v4_eng-html5.html

[10] B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, Sep. 2000. [Online]. Available: https://www.rfc-editor.org/info/rfc2898/

[11] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: The memory-hard function for password hashing and other applications," March 2017, version 1.3 of Argon2: PHC release. [Online]. Available: https://github.com/P-H-C/phc-winner-argon2/blob/master/argon2-specs.pdf

[12] D. Anton and E. Simion, “Linux unified key setup (luks) - the good, the bad, the ugly,” 04 2019.

[13] W. Arthur, D. Challener, and K. Goldman, *A Practical Guide to TPM 2.0*. Apress, 2015.

[14] pi3g, “From application to tpm - linux deepdive,” https://www.youtube.com/watch?v=PF5Uv16kJ-M, October 2025, accessed: August 8, 2026.